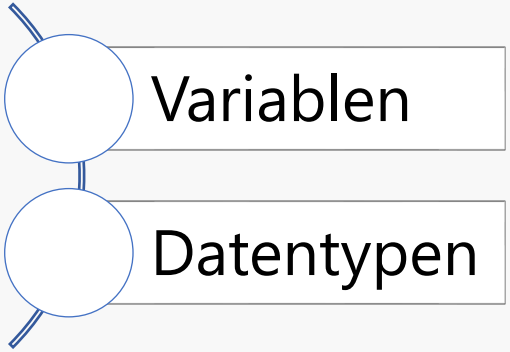
Operatoren – ein erster Überblick

Einführung in die Programmierung 1

TU Wien, Wintersemester 25

Was wurde zuletzt besprochen?



Was wird in diesem Foliensatz besprochen?

- Operatoren
- Ausdrücke / Anweisungen
- Typanpassung

Was lernen Sie kennen?

- Grundlegenden Operatoren in Java
- Implementierung von Ausdrücken bzw. Anweisungen in Java
- Typanpassung in Java

Operatoren

Über Operatoren erhalten wir die Möglichkeit
• Variablen mit Inhalten zu belegen oder • die in ihnen gespeicherten Werte zu verändern

Operatoren – Operandenanzahl

Unäre Operatoren (ein Operand)	• Beispiel für Form: **-**x
Binäre Operatoren (zwei Operanden)	• Beispiel für Form: a **+** b
Ternäre Operatoren (drei Operanden)	• Beispiel für Form: evaluation **?** firstResult **:** secondResult

Operatoren – Position

Präfixform	• Operator steht vor seinem bzw. seinen Operanden • Beispiel: ++counter
Postfixform	• Operator steht hinter seinem bzw. seinen Operanden • Beispiel: counter++
Infixform	• Operator steht zwischen seinen Operanden (binäre Operatoren) • Beispiel: a + b

Operatoren – Auswertungsreihenfolge

Linksassoziativ

- Auswertung von links nach rechts für einen Operator ◊:
  - X ◊ Y ◊ Z entspricht (X ◊ Y) ◊ Z
- z. B.
  - 3 + 4 + 5 entspricht (3 + 4) + 5
  - 128 / 16 / 4 entspricht (128 / 16) / 4

Rechtsassoziativ

- Auswertung von rechts nach links für einen Operator ◊:
  - X ◊ Y ◊ Z entspricht X ◊ (Y ◊ Z)
- Beispiel siehe nächste Folie

Zuweisungsoperator =

Binärer Operator

- Beispiele

```
int a;
a = 1;
int b = 3;
```

Wert muss nicht konstant sein

- Beispiel

```
int a = 1;
int b = a;
```

Rechtsassoziativ

- Beispiel

```
int a, b;
a = b = 1;
```

Nach Ausführung haben a und b den Wert 1!

Arithmetische Operatoren (Auswahl)

Name	Verwendung	Operator liefert
Minus (unär)	- Op1	Vorzeichenwechsel
Minus (binär)	Op1 - Op2	Differenz
Plus	Op1 + Op2	Summe
Multiplikation	Op1 * Op2	Produkt
Division	Op1 / Op2	Ganzzahlige Division bei ganzen Zahlen (Nachkommaanteil wird abgeschnitten), Quotient
Modulo	Op1 % Op2	Rest bei Division

Linksassoziativ wenn binär, rechtsassoziativ wenn unär

Beispiel (ganzzahlige Berechnungen)

```java
public class Operators {

    public static void main(String[] args) {
        int x = 10;
        int y = 20;
        int z = 30;
        int a = x * y;
        int b = a + -z - 15;
        System.out.println(a + " " + b);
        a = b / x;
        b = b % x;
        System.out.println(a + " " + b);
    }
}
```

```
200 155
15 5
```

Verkürzte Schreibweise (allgemeine Form)

Operation	Bezeichnung	entspricht
Op1 += Op2	Additionszuweisung	Op1 = Op1 + Op2
Op1 -= Op2	Subtraktionszuweisung	Op1 = Op1 - Op2
Op1 *= Op2	Multiplikationszuweisung	Op1 = Op1 * Op2
Op1 /= Op2	Divisionszuweisung	Op1 = Op1 / Op2
Op1 %= Op2	Modulozuweisung	Op1 = Op1 % Op2

Verkürzte Schreibweise – Beispiele

Gegebene Deklarationen (ganze Zahlen)

```
c = 3;
d = 5;
e = 4;
f = 6;
g = 12;
```

Operator	Ausdruck	Erklärung	Ergebnis
+=	c += 7	c = c + 7	10
-=	d -= 4	d = d - 4	1
*=	e *= 5	e = e * 5	20
/=	f /= 3	f = f / 3	2
%=	g %= 9	g = g % 9	3

Inkrement und Dekrement

Operator	Benennung	Beispiel für Ausdruck	Erklärung	Ergebnis des Ausdrucks
++	Präinkrement	++a	a wird um 1 erhöht	a + 1
++	Postinkrement	a++	a wird um 1 erhöht	a
--	Prädekrement	--b	b wird um 1 verringert	b - 1
--	Postdekrement	b--	b wird um 1 verringert	b

Beispiel (Inkrementierung)

```java
public class IncrementTest {

    public static void main(String[] args) {
        int a = 0, b = 0, c = 0;
        System.out.println("a = " + a + ", b = " + b + ", c = " + c);
        a = 3;
        System.out.println("a = " + a + ", b = " + b + ", c = " + c);
        b = ++a;
        System.out.println("a = " + a + ", b = " + b + ", c = " + c);
        c = a++;
        System.out.println("a = " + a + ", b = " + b + ", c = " + c);
    }

}
```

```
a = 0, b = 0, c = 0
a = 3, b = 0, c = 0
a = 4, b = 4, c = 0
a = 5, b = 4, c = 4
```

Ganze Zahlen – Endlichkeit

- Keine Anzeige eines Überlaufs
- Beispiel

```
int x, y;
x = Integer.MAX_VALUE; // x = 2147483647
y = x + 1; // y = -2147483648 = Integer.MIN_VALUE
y = x * 2; // -2 !!!
```

Konstante **MAX_VALUE** kommt aus in der Klasse Integer
Hinweis: Zu jedem einfachen Datentyp gibt es eine entsprechende Klasse (in diesem Beispiel die Klasse Integer)

Ganze Zahlen – Eigenarten

- Bei additiven Operationen kann über die Grenzen hinaus und wieder zurück gerechnet werden
- Bei multiplikativen Operationen ist dabei Vorsicht geboten

Ganze Zahlen – Abbruch

Division durch 0 nicht erlaubt

Sofortiger Programmabbruch

- Es wird eine **Ausnahme** (Exception) geworfen
- Z. B. Ausgabe in IntelliJ
  - Exception in thread "main" java.lang.ArithmeticException: / by zero …
- Nicht fortfahren und „Ergebnis“ ignorieren

Gleitkommazahlen – Größe und Genauigkeit

- Nicht alle Zahlen können genau dargestellt werden (z. B. 0.1)
- Berechnungen führen immer wieder zu „falschen“ Ergebnissen, z.B.

```
System.out.println(0.1 + 0.2); // 0.30000000000000004

System.out.println(0.1 + 0.3); // 0.4
```

Gleitkommazahlen – Probleme bei unterschiedlich großen Zahlen

```
double epsilon = 1E-14;
double x = 10.0 + epsilon;
double y = epsilon / (x - 10.0);
```

y sollte 1 sein - obiger Code ergibt aber 0.9382499223688533!

Ausdruck (expression)

Auswertung ergibt einen einzigen wohl definierten Wert

Kann Teil eines größeren Ausdrucks sein

Einfache Form

- Konstante, Zahl etc.

Komplexere Form

- Kombination von Operatoren und Operanden
- Beispiel: 3 + 4 erzeugt den Wert 7

Anweisung (statement)

Einzelne Vorschrift, die im Rahmen der Abarbeitung des Programms auszuführen ist

Bisherige Beispiele

- Deklarationen
- Durch Semikolon abgeschlossene Ausdrücke, z. B. x = x + y;

Anweisungen haben keinen Wert

- Sie können nicht Teil eines größeren Ausdrucks sein
- Sie können aber **Seiteneffekte** haben
  - **i++;** möglich und sinnvoll (i wird um 1 erhöht)

Vorrang bei unterschiedlichen Operatoren

Z. B. „Punkt- vor Strichrechnung“

Allgemein für Operatoren ◊ und ● - wie wird X ◊ Y ● Z ausgewertet?

(X ◊ Y) ● Z -> ◊ hat den höheren Rang

X ◊ (Y ● Z) -> ● hat den höheren Rang

Operatorvorrang allgemein

Operatorvorrang regelt die implizite Klammerung bei Operatoren auf **verschiedenen** Stufen

Zu unterscheiden von Assoziativität

- Regelt die implizite Klammerung bei Operatoren auf **gleicher** Stufe

Operatorvorrang – Beispiel

x = 2 + 4 * 5;

Operatoren: *, +, =
(geordnet nach Vorrang)

Eigentlich x = 2 + (4 * 5) da „Punkt- vor Strichrechnung“

4 * 5 auswerten

2 + 20 berechnen

22 der Variable x zuweisen

Operanden und Operatoren

Operand wird vom Operator mit höherem Rang verwendet

Wenn zwei Operatoren denselben Rang haben, dann entscheidet die Assoziativität

Wenn zwei Operatoren denselben Rang und dieselbe Assoziativität haben, dann werden die (Teil-)Ausdrücke von links nach rechts ausgewertet

Eigene Klammerung

- Wenn wir aber etwas anderes wollen?
  - Klammern verwenden!
- Beispiel

$$3 * 4 + 5 * 7 = 12 + 35 = 47$$

$$3 * (4 + 5) * 7 = 3 * 9 * 7 = 189$$

Ausdrücke und unterschiedliche Datentypen

Variablen unterschiedlichen Typs in einem Ausdruck

- Welchen Typ hat der Ausdruck?

Für alle arithmetischen Operationen gilt

- Der „kleinere“ Operand wird vor der Ausführung der Operation in den „größeren“ konvertiert
- Der Ausdruck bekommt dann den gleichen Typ wie seine Operanden
- Der kleinste Typ ist aber zumindest **int**

Unterschiedliche Typen – Beispiel

Beispiel

```
int a = 1234567;
long b = 12345678L;
long c = a + b;
```

Erklärung

- a ist vom Typ int
- b ist vom Typ long
- a + b ist vom Typ long

Typkompatibilität (1)

Operanden eines Ausdrucks müssen kompatibel sein

Beispiel für ein Problem (Fehler) mit inkompatiblen Typen

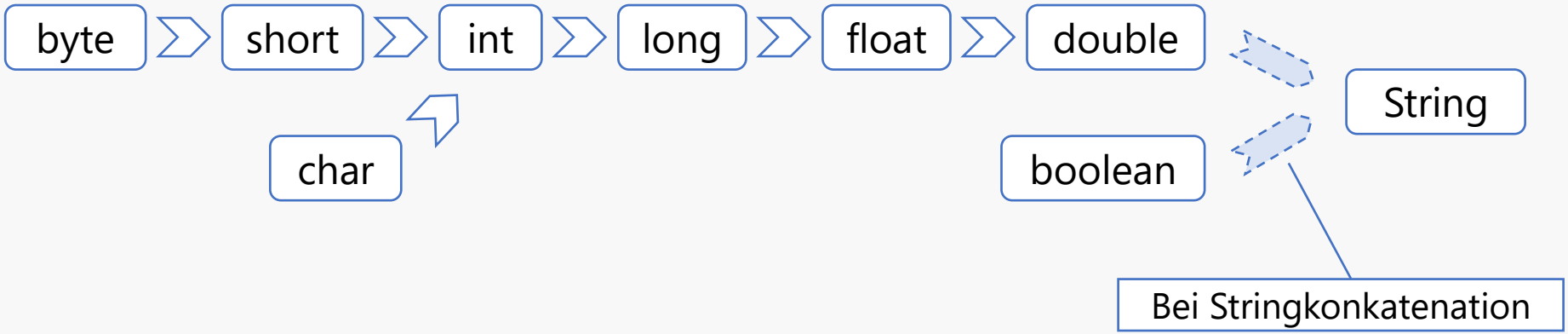
```
int x;
x = 2.5;
```

Typkompatibilität (1)

Zwei Typen sind kompatibel, wenn
• sie gleich sind, • wenn sie durch implizite Typanpassung zum gleichen Typ führen

Implizite Typanpassung

- Automatische Typanpassung
- Nur in eine Richtung (**vereinfacht**)



Explizite Typanpassung (Cast)

- Form

```
(Zieltyp) Ausdruck
```

- Beispiel von vorher

```
int x;
x = (int) 2.5;
```

- Kann zu Datenverlusten führen – Beispiel

```
int x = 300;          // 00000000 00000000 00000001 00101100
byte y = (byte) x;    // 00101100
System.out.println(y);
```

44

Beispiel (Typanpassung)

```java
public class TypeTest {
    public static void main(String[] args) {
        short s = 10;
        float f = 100.5f;
        double d = 200.1, sum;
        boolean flag = true;
        char character = 'x';
        String header = "Results:";
        System.out.println(header);
        System.out.println("s = " + s + " f = " + f + " d = " + d);
        sum = s + f + d;
        System.out.println(flag);
        System.out.println(character);
        System.out.println("sum = " + sum);
        System.out.println(2 + "test" + 3);
        System.out.println(2 + 3 + "test");
        System.out.println("test" + 2 + 3);
        System.out.println("test" + 2 * 3);
    }
}
```

```
Results:
s = 10 f = 100.5 d = 200.1
true
x
sum = 310.6
2test3
5test
test23
test6
```

Local Variable Type Inference (ab Java 10)

Keine explizite Typangabe auf der linken Seite einer Variablendeklaration

Compiler muss anhand der rechten Seite den konkreten Typ ermitteln können

Local Variable Type Inference – Beispiel

```
public class VarTest {
    public static void main(String[] args) {
        var a = 10;
        var b = 12.5;
        var c = "Hello";
        var d = Integer.MAX_VALUE;
        var e = Long.parseLong("123");
        System.out.println(a + " " + b + " " + c + " " + d + " " + e);
    }
}
```

Methode, die einen Wert vom Typ long zurückliefert

```
10 12.5 Hello 2147483647 123
```